Lesson:

HashMap

Pre Requisites:

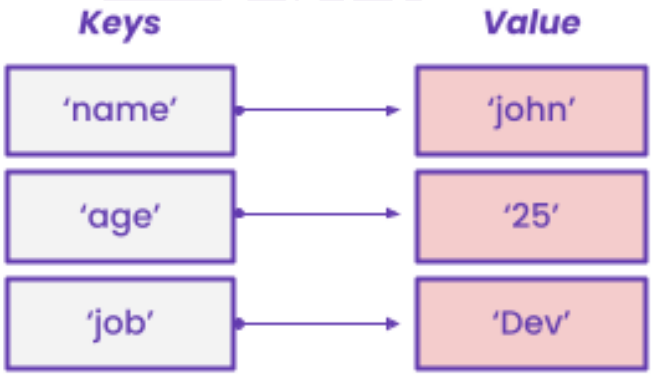
- Stack and Queues
- Collection Framework in Java

List of concepts involved :

- What is a HashMap
- Introduction to HashMap class
- HashMap Hierarchy
- Constructors for a HashMap class
- How to work with HashMaps
- Important features of HashMap class
- Applications of HashMaps
- Internal Working of a HashMap: Hashing
- Hash Functions
- Collisions
- Collision Resolution Techniques
- Load Factor

Topic: What is a HashMap?

- It is a data structure that implements an associative array or dictionary.
- An abstract data type that maps keys to values.
- One object is used as a key (index) to another object (value).
- All major operations on a HashMap are achieved in O(1) time complexity.

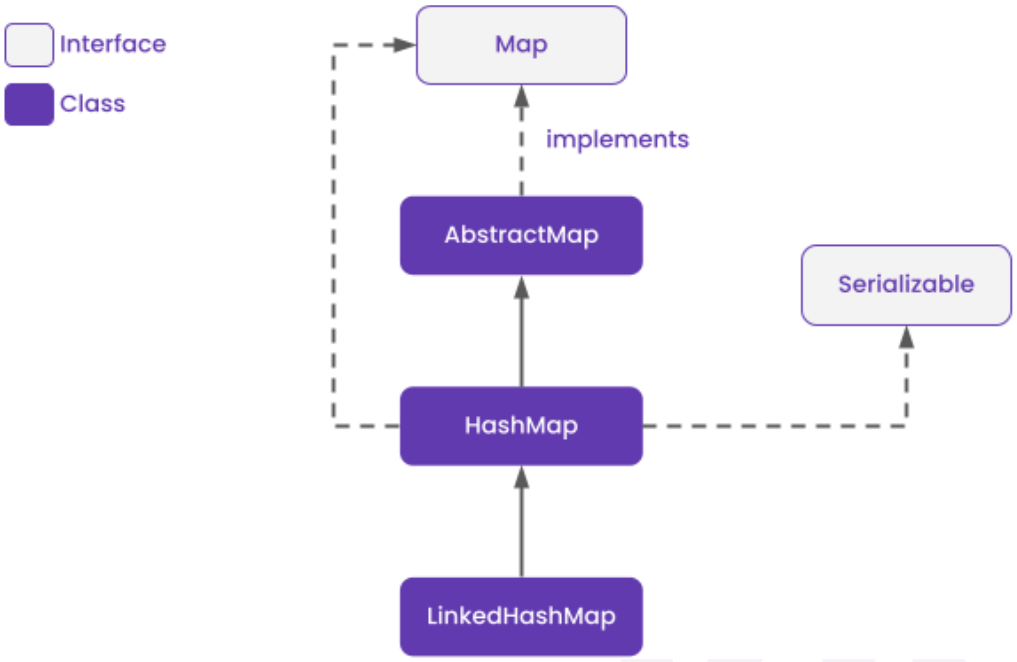


Topic: Introduction to HashMap class

- The HashMap class of the Java collections framework provides the functionality of the hash table data structure.
- HashMap class is found in the java.util package.
- Java HashMap class implements the Map interface which allows us to store key and value pair, where keys should be unique.
- Here, keys are unique identifiers used to associate each value on a map.

- If you try to insert the duplicate key, it will replace the element of the corresponding key.
- To use this class and its methods, you need to import the java.util.HashMap package or its superclass.

Topic: HashMap Hierarchy



- HashMap class inherits the AbstractMap class and implements the Map interface.
- LinkedHashMap is its direct subclass.
- HashMap implements Serializable, Cloneable, Map<K, V> interfaces.

Topic: Constructors for a HashMap Class:

Constructor	Description
HashMap()	It is used to construct a default HashMap.
HashMap(Map<? extends K,? extends V> m)	It is used to initialize the hash map by using the elements of the given Map object m.
HashMap(int capacity)	It is used to initializes the capacity of the hash map to the given integer value, capacity.
HashMap(int capacity, float loadFactor)	It is used to initialize both the capacity and load factor of the hash map by using its arguments.

Topic: How to work with HashMaps

- **Syntax**

```
HashMap<K, V> hm = new HashMap<K, V>();
```

Example: Create a hashmap that stores Integer values for Character keys.
https://pastebin.com/yfcsDD1g

- **Adding Elements:**

To store unique key-value pairs, we will use put() method.
https://pastebin.com/XkVnukQc

Time Complexity: O(1)

- **To get the value of an already present key in the hashmap:**

We will use get() method, this will give value for the key.

https://pastebin.com/zfbYvwd3

```
/Library/Java/JavaVirtualMachines/jdk-1
2

Process finished with exit code 0
```

Time complexity: O(1)

Consider a case when the key we are trying to get the value of is not present in the map.
Let's remove 'b' from the map and then try to get its value:
https://pastebin.com/2qrzwe8R

```
/Library/Java/JavaVirtualMachines/jdk-1
null

Process finished with exit code 0
```

Here, we get null as output as key 'b' does not exist in the hashmap. To avoid this, always check if the key is present in the hashmap, and then get its value.
https://pastebin.com/dxSX6f67

```
/Library/Java/JavaVirtualMachines/jdk-1
2

Process finished with exit code 0
```

- **Changing value of an already present key in the hashmap:**

We will use put() method, this will update the newly inserted value for the same key.
https://pastebin.com/AkbZJD7z

```
/Library/Java/JavaVirtualMachines/jdk-1
3

Process finished with exit code 0
```

Time complexity: O(1)

- **To check if a key is already present in the hashmap:**

We will use containsKey() method, this will give boolean answer.
https://pastebin.com/fgSvfPs4

```
/Library/Java/JavaVirtualMachines/jdk-1
Yes

Process finished with exit code 0
```

Time complexity: O(1)

- **Removing a pair from the HashMap**

We will use remove() method, this will remove the key and the mapping for it if present in the hashmap.
https://pastebin.com/CGHB1iBG

Time complexity: O(1)

- **Size of the HashMap**

We will use size() method, this will return the size or the number of entries present in the hashmap.
https://pastebin.com/5HbaHuWX

```
/Library/Java/JavaVirtualMachines/jdk-1
3

Process finished with exit code 0
```

Time complexity: O(1)

- **Traversing a HashMap**

To view all key-value pairs, we use Entry< ? , ? >
https://pastebin.com/JAh2vAfu

```
/Library/Java/JavaVirtualMachines/jdk-19
Key: a Value: 3
Key: c Value: 4
Key: d Value: 7

Process finished with exit code 0
```

Time complexity: O(n) where n is the size of hashmap.

To view all keys or the set of keys, we use keySet().
https://pastebin.com/Ue0X2kNw

```
/Library/Java/JavaVirtualMachines/jdk-19
a
c
d

Process finished with exit code 0
```

Time complexity: O(n) where n is the size of hashmap.

To view all values or the set of values, we can also use keySet() in a different manner.
https://pastebin.com/hh2rtfgg

```
/Library/Java/JavaVirtualMachines/jdk-19
3
4
7

Process finished with exit code 0
```

Time complexity: O(n) where n is the size of hashmap.

Or we can use values().
https://pastebin.com/ZnYkr2zg

```
/Library/Java/JavaVirtualMachines/jdk-19
3
4
7

Process finished with exit code 0
```

Time complexity: O(n) where n is the size of hashmap.
Example: Create a HashMap using Java HashMap class to store the following pairs(Person, Age) and display them.
Input:
Akash 21
Yash 16
Lav 17
Rishika 19
Harry 18

Output:
Age of Akash is 21
Age of Yash is 16
Age of Lav is 17
Age of Rishika is 19
Age of Harry is 18

Explanation:

- Since, pairs are of Person, age; we need to store String, Integer pairs.
- To use HashMap class, we need to import the class. Here we have imported all the classes of import.java.util. package.
- We will manually put each pair one by one and then display them using methods of Java HashMap class.

Code:
https://pastebin.com/mcLL95Z0

```
/Library/Java/JavaVirtualMachines/jdk-19.jd
Age of Lav is 17
Age of Rishika is 19
Age of Yash is 16
Age of Harry is 18
Age of Akash is 21

Process finished with exit code 0
```

Topic: Important features of HashMap class

- To access a value one must know its key.
- HashMap doesn't allow duplicate keys but allows duplicate values. That means A single key can't contain more than 1 value but more than 1 key can contain a single value.
- HashMap allows null key also but only once and multiple null values.
- Java HashMap maintains no order.

Topic: Applications of HashMaps

- Problems related to frequency of an item: Hash maps are useful for solving most problems related to item frequency. An example of such a problem is finding common characters in a list of strings. If an item appears twice in all strings, it needs to be included twice in the output. To solve this problem, we can use two for loops to compare one string with a list of other strings. But the time complexity of such an approach is $O(n^2)$. To minimize the time complexity, we can use a hash map to store the characters as keys with the string number and frequency as a pair for value. We can then iterate the hash map to check for common characters found in all strings along with their frequency. This reduces the time complexity to $O(n)$.

- Mapping problems: Hash maps are used to link the value with the key. To store the correspondence between the any item and its related value, we use a map.

- Storage optimization: Dropbox uses hash tables for storage optimization in the cloud. Suppose there are multiple users who upload the same video to their Dropbox accounts. Since storing multiple copies of the same video will consume too much storage, Dropbox stores a single copy of the file with the link provided to each user for access. When a new user uploads the same file, Dropbox computes the hash value for the file. If it matches the value stored in the hash table, the user is returned the link to the file.

- Dictionary: If we were to make an online portal for searching the meaning of a word, we would be building a dictionary in the back end to store the word along with its meaning. When a user searches for a word, it uses the find hash function to retrieve its meaning.

- Phonebook: We can use HashMap for the implementation of Phonebook by using key-value pair of Name - Number.

PROBLEM

Question: Given an array, find the most frequent element in it. If there are multiple elements that appear a maximum number of times, print any one of them.

Input 1:
n = 6
arr[] = {1, 3, 2, 1, 4, 1}
Output 1:
1
Input 2:
n = 7
arr[] = {10, 20, 10, 20, 30, 20, 20}
Output 2:
20

Explanation

- Create a HashMap that stores an Integer to Integer key-value pairs.
- Traverse over the array.
- For every element in the array, check if it is already present in the HashMap or not.
- If it is already present, it means we have found another duplicate of the same element, and thus we need to increment its frequency which is denoted by its value. So get the value of this key, increment by 1 and put it back.
- If the element is not present in the HashMap, put it in the HashMap as the key and its value being 1, which denotes that this is the first appearance of this element.
- After traversing over the entire array, we will now traverse over the HashMap.
- We traverse over the keys, and check if its value is greater than the current max pointers value which has been intialized with 0.
- If it is greater, update the max_element pointer with the current key as well.
- In the end, print the max_element.

Code:
https://pastebin.com/b8pm3MYE

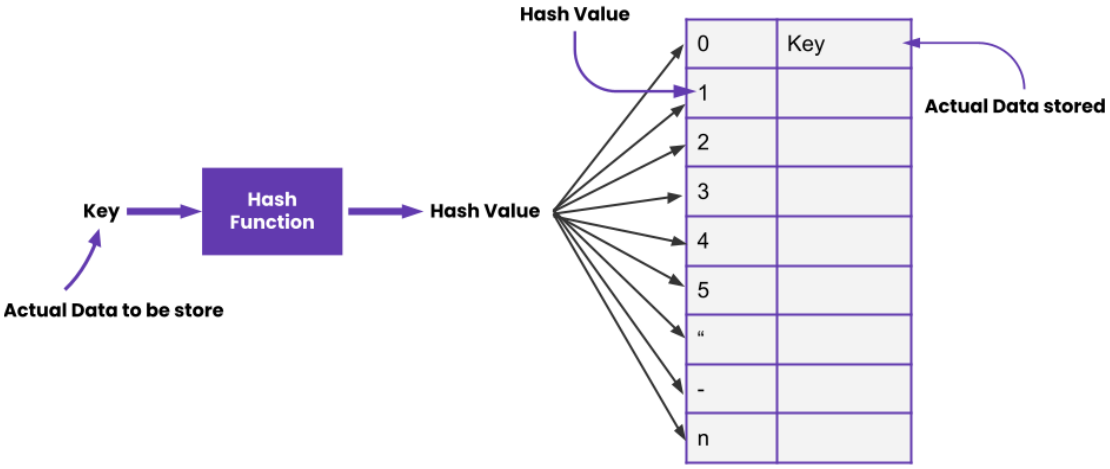
```
/Library/Java/JavaVirtualMachines/jdk-19.jdk/Conten
Enter the length of the array:
6
Enter the elements of the array:
1 3 2 1 4 1
1

Process finished with exit code 0
```

Topic: Internal Working of a HashMap: HASHING

- Hashing is a technique of mapping keys, and values into the hash table by using a hash function.

- A real life example of hashing is giving a name or a nickname to a person. It helps in uniquely identifying that person every time.
- It is the process of designing a Hash function which takes an input and gives a consistent output.
- Hash table is a data structure used to store key-value pairs.
- It helps in faster access of elements.
- The efficiency of mapping depends upon the efficiency of the hash function used.



- When we talk about key-value pairs inside the hash table, the key represents the hash value and the value represents the actual data.
- HashMaps are a default implementation of HashTables which takes amortized O(1) time to access the keys and their corresponding values.

Topic: What are Hash Functions?

- These are mathematical formulas that create a mapping between key and value.
- A hash function that maps every item into its own unique slot is known as a perfect hash function.
- A good hash function is efficiently computable, uniformly distributes the keys and minimizes collisions.
- We can create our own hash function as well.
- Based on these qualities, we generally use these 4 types of hash functions:

1. Division Method: divide the value k by M and then use the remainder obtained.
   h(K) = k mod M
   Example: k = 1276, M = 11
   h(1276) = 1276 mod 11 = 0

2. Mid Square Method: Square the value of the key k and extract the middle r digits as the hash value.
   h(K) = h(k x k)
   k = 60
   k x k = 60 x 60 = 3600
   h(60) = 60

3. Digit Folding Method: Divide the key-value k into a number of parts i.e. k1, k2, k3,....,kn, where each part has the same number of digits except for the last part that can have lesser digits than the other parts. Add the individual parts. The hash value is obtained by ignoring the last carry if any.
   k = k1, k2, k3, k4, ..., kn, s = k1+ k2 + k3 + k4 +....+ kn, h(K)= s

k = 12345
k1 = 12, k2 = 34, k3 = 5
s = k1 + k2 + k3 = 12 + 34 + 5 = 51
h(K) = 51

4. Multiplication Method: Choose a constant value A such that 0 < A < 1. Multiply the key value with A. Extract the fractional part of kA. Multiply the result of the above step by the size of the hash table i.e. M. The resulting hash value is obtained by taking the floor of the result obtained
   h(K) = floor (M (kA mod 1))
   k = 12345
   A = 0.357840
   M = 100
   h(12345) = floor[100 (12345*0.357840 mod 1)]
   = floor[100 (4417.5348 mod 1)]
   = floor[100 (0.5348)]
   = floor[53.48]
   = 53

Topic: Collisions

- Sometimes, the same Hash function can result in the same hash value for different keys which is also known as a collision.
- For example, consider the hash function: h(k) = k % M where M = 2. This will give the same hash value for every odd number = 1 and every even number = 0: h(5) = 1, h(7) = 1 and h(6) = 0, h(8) = 0.
- The hashing process generates a small number for a big key, so there is a possibility that two keys could produce the same value.
- This is known as collision and it creates problem in searching, insertion, deletion, and updating of value.
- The situation where the newly inserted key maps to an already occupied hash value must be handled using some collision handling technology.

Topic: Collision Resolution Techniques

There are mainly two methods to handle collision:

- Separate Chaining (Open Hashing): It makes each cell of the hash table point to a linked list of records that have the same hash function value. Chaining is simple but requires additional memory outside the table.
- Open Addressing (Closed Hashing): All elements are stored in the hash table itself. Each table entry contains either a record or NIL. When searching for an element, we examine the table slots one by one until the desired element is found or it is clear that the element is not in the table.
- It further is done in three ways:
  a. Linear Probing: Searching the hash table sequentially that starts from the original location of the hash until an empty location is found.
  b. Quadratic Probing: Taking the original hash index and adding successive values of an arbitrary quadratic polynomial until an open slot is found.
  c. Double Hashing: It makes use of two hash function. The first hash function is h1(k) which takes the key and gives out a location on the hash table. But if the new location is not occupied or empty then we can easily place our key.
- But in case the location is occupied (collision) we will use secondary hash-function h2(k) in combination with the first hash-function h1(k) to find the new location on the hash table.

- h(k, i) = (h1(k) + i * h2(k)) % n
- Here, i represents the collision number.

Topic: Load Factor

- The load factor of the hash table can be defined as the number of items the hash table contains divided by the size of the hash table.
- Similarly, threshold value is the product of load factor and current capacity of the hash table.
- Load factor is the decisive parameter that is used when we want to rehash(hashing again) the previous hash function or want to add more elements to the existing hash table.
- It helps us in determining the efficiency of the hash function i.e. it tells whether the hash function which we are using is distributing the keys uniformly or not in the hash table.
-

Example
Question: Implement your own HashMap with the following methods:

- put(key, value): Returns void or inserts/updates
- get(key): Returns value of the key if exists or returns null
- size(): Returns number of entries in the HashMap
- remove(key): Removes entry with specified key from the HashMap and returns value/null
- Both keys and value can be of any type

Code:
https://pastebin.com/iM3G7Fcq

Output:

```
"C:\Program Files\Java\jdk-14.0.1\
Testing put
Testing size 3
Testing values 1
Testing values 2
Testing values 30
Testing values null
Testing remove 30
Testing remove null
Testing size 2

Process finished with exit code 0
```

Upcoming Class Teasers:

- HashMaps-2